POSIX Threads (pthreads) Cheat Sheet

Complete Reference for Multithreaded Programming in C | `#include <pthread.h>` | Compile: `gcc -pthread`

Data Types

`pthread_t`	Thread identifier
`pthread_attr_t`	Thread attributes
`pthread_mutex_t`	Mutex lock
`pthread_mutexattr_t`	Mutex attributes
`pthread_cond_t`	Condition variable
`pthread_condattr_t`	Condition attributes
`pthread_rwlock_t`	Read-write lock
`pthread_barrier_t`	Barrier sync
`pthread_spinlock_t`	Spinlock
`pthread_key_t`	Thread-local key
`pthread_once_t`	Once control

Thread Management

Create & Join

```c
int pthread_create(
    pthread_t *thread,
    const pthread_attr_t *attr,
    void *(*start_routine)(void*),
    void *arg
);

int pthread_join(
    pthread_t thread,
    void **retval  // Get return value
);
```

Other Functions

`pthread_self()`	Get current thread ID
`pthread_equal(t1,t2)`	Compare thread IDs
`pthread_detach(t)`	Auto-cleanup on exit
`pthread_exit(ret)`	Exit current thread

Tip

Detached threads free resources automatically.
Joinable (default) must be joined to avoid memory leaks.

Basic Example

```c
void* worker(void* arg) {
    int id = *(int*)arg;
    printf("Thread %d running\n", id);
    return NULL;
}

int main() {
    pthread_t t;
    int arg = 42;
    pthread_create(&t, NULL, worker, &arg);
    pthread_join(t, NULL);
    return 0;
}
```

Thread Attributes

```c
pthread_attr_t attr;
pthread_attr_init(&attr);

// Detach state
pthread_attr_setdetachstate(&attr,
    PTHREAD_CREATE_DETACHED);
    // or PTHREAD_CREATE_JOINABLE

// Stack size (e.g., 2MB)
pthread_attr_setstacksize(&attr,
    2 * 1024 * 1024);

// Scheduling policy
pthread_attr_setschedpolicy(&attr,
    SCHED_FIFO);  // or SCHED_RR

pthread_attr_destroy(&attr);
```

Mutexes

Basic Operations

```c
// Static initialization
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

// Dynamic initialization
pthread_mutex_init(&m, NULL);

// Lock operations
pthread_mutex_lock(&m);      // Blocking
pthread_mutex_trylock(&m);   // Non-blocking
pthread_mutex_unlock(&m);    // Release

// Cleanup
pthread_mutex_destroy(&m);
```

Timed Lock

```c
struct timespec ts;
clock_gettime(CLOCK_REALTIME, &ts);
ts.tv_sec += 5;  // 5 second timeout

int rc = pthread_mutex_timedlock(&m, &ts);
if (rc == ETIMEDOUT) {
    // Handle timeout
}
```

Mutex Types

`NORMAL`	Deadlock if re-locked
`ERRORCHECK`	Returns error on re-lock
`RECURSIVE`	Allows recursive locking

```c
pthread_mutexattr_t attr;
pthread_mutexattr_init(&attr);
pthread_mutexattr_settype(&attr,
    PTHREAD_MUTEX_RECURSIVE);
```

```
pthread_mutex_init(&m, &attr);
pthread_mutexattr_destroy(&attr);
```

Warning

Always unlock in the **same thread** that locked!
Never unlock a mutex you don't own.

Condition Variables

Basic Operations

```
// Static initialization
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

// Wait (atomically releases mutex)
pthread_cond_wait(&c, &mutex);

// Wait with timeout
pthread_cond_timedwait(&c, &mutex, &ts);

// Signal one waiting thread
pthread_cond_signal(&c);

// Signal ALL waiting threads
pthread_cond_broadcast(&c);

// Cleanup
pthread_cond_destroy(&c);
```

Correct Usage Pattern

```
// === WAITER ===
pthread_mutex_lock(&mutex);
while (!condition) {    // Always WHILE!
    pthread_cond_wait(&cond, &mutex);
}
// ... do protected work ...
pthread_mutex_unlock(&mutex);

// === SIGNALER ===
pthread_mutex_lock(&mutex);
condition = true;
pthread_cond_signal(&cond);
pthread_mutex_unlock(&mutex);
```

Warning

Always use `while`, never `if`!
Spurious wakeups can and do occur.

Signal vs Broadcast

`signal`	Wake **one** waiting thread
`broadcast`	Wake **all** waiting threads

Use `broadcast` for: shutdown signals, state changes affecting multiple threads.

Read-Write Locks

Multiple concurrent readers **OR** single exclusive writer.

```
pthread_rwlock_t rw =
    PTHREAD_RWLOCK_INITIALIZER;

// Read lock (shared access)
pthread_rwlock_rdlock(&rw);
pthread_rwlock_tryrdlock(&rw);

// Write lock (exclusive access)
pthread_rwlock_wrlock(&rw);
pthread_rwlock_trywrlock(&rw);

// Unlock (same for both)
pthread_rwlock_unlock(&rw);

pthread_rwlock_destroy(&rw);
```

Tip

Best when reads » writes. For balanced read/write, use a simple mutex instead.

Barriers

Synchronize exactly N threads at a rendezvous point.

```
pthread_barrier_t b;

// Initialize for N threads
pthread_barrier_init(&b, NULL, N);

// Wait (blocks until N threads arrive)
int rc = pthread_barrier_wait(&b);
if (rc == PTHREAD_BARRIER_SERIAL_THREAD) {
    // Exactly ONE thread gets this
    // Use for cleanup, etc.
}

pthread_barrier_destroy(&b);
```

Spinlocks

Busy-wait lock for *extremely short* critical sections.

```
pthread_spinlock_t s;

pthread_spin_init(&s,
    PTHREAD_PROCESS_PRIVATE);

pthread_spin_lock(&s);     // Busy waits!
pthread_spin_trylock(&s); // Non-blocking
pthread_spin_unlock(&s);

pthread_spin_destroy(&s);
```

Warning

Burns CPU cycles while waiting! Only use for lock hold times < context switch time.

Thread-Local Storage

pthread_key API

```
pthread_key_t key;

// Create key with destructor (once)
pthread_key_create(&key, destructor_fn);

// Set value for THIS thread only
pthread_setspecific(key, value);

// Get value for THIS thread
void* val = pthread_getspecific(key);

pthread_key_delete(key);
```

__thread Keyword (GCC/Clang)

```
__thread int counter = 0;
__thread void* my_data = NULL;

// Each thread has its own copy!
// Simpler than pthread_key API
```

Once Initialization

Execute initialization code exactly once, thread-safely.

```
pthread_once_t once = PTHREAD_ONCE_INIT;

void init_function(void) {
    // This runs exactly ONCE
    // regardless of how many threads call
}

// Called from any/all threads:
pthread_once(&once, init_function);
```

Thread Cancellation

```
// Request thread cancellation
pthread_cancel(thread);

// Enable/disable cancellation
pthread_setcancelstate(
    PTHREAD_CANCEL_ENABLE, NULL);
    // or PTHREAD_CANCEL_DISABLE

// Set cancellation type
pthread_setcanceltype(
    PTHREAD_CANCEL_DEFERRED, NULL);
    // or PTHREAD_CANCEL_ASYNCHRONOUS

// Create explicit cancellation point
pthread_testcancel();
```

Cleanup Handlers

```
pthread_cleanup_push(cleanup_fn, arg);
// ... cancellable code ...
pthread_cleanup_pop(1);  // 1 = execute handler
```

Semaphores

```
#include <semaphore.h>
```

```
sem_t sem;

// Initialize with count
sem_init(&sem, 0, initial_value);

// Wait (decrement, blocks if 0)
sem_wait(&sem);
sem_trywait(&sem);
sem_timedwait(&sem, &ts);

// Post (increment, wake waiters)
sem_post(&sem);

// Get current value
sem_getvalue(&sem, &val);

sem_destroy(&sem);
```

Tip

Mutex = binary semaphore (0 or 1)
Semaphore = counting (0 to N)

Error Codes

EAGAIN	Insufficient resources
EBUSY	Resource is busy
EDEADLK	Deadlock detected
EINVAL	Invalid argument
ENOMEM	Out of memory
EPERM	Operation not permitted
ESRCH	No such thread
ETIMEDOUT	Operation timed out

```
int rc = pthread_create(&t, NULL, fn, arg);
if (rc != 0) {
    fprintf(stderr, "Error: %s\n",
            strerror(rc));
    exit(EXIT_FAILURE);
}
```

Scheduling

```
// Get/Set scheduling parameters
pthread_getschedparam(t, &policy, &param);
pthread_setschedparam(t, policy, &param);

// Set thread priority
pthread_setschedprio(t, priority);

// Yield CPU to other threads
sched_yield();

// Get priority range for policy
int min = sched_get_priority_min(policy);
int max = sched_get_priority_max(policy);
```

Scheduling Policies

SCHED_OTHER	Default time-sharing
SCHED_FIFO	Real-time FIFO
SCHED_RR	Real-time round-robin

Common Patterns

Producer-Consumer Queue

```
// === PRODUCER ===
pthread_mutex_lock(&mutex);
while (queue_full)
    pthread_cond_wait(&not_full, &mutex);
enqueue(item);
pthread_cond_signal(&not_empty);
pthread_mutex_unlock(&mutex);

// === CONSUMER ===
pthread_mutex_lock(&mutex);
while (queue_empty)
    pthread_cond_wait(&not_empty, &mutex);
item = dequeue();
pthread_cond_signal(&not_full);
pthread_mutex_unlock(&mutex);
```

Thread Pool Worker

```
void* worker(void* pool_ptr) {
    while (1) {
        pthread_mutex_lock(&mutex);
        while (queue_empty && !shutdown)
            pthread_cond_wait(&job_ready, &mutex);

        if (shutdown && queue_empty) {
            pthread_mutex_unlock(&mutex);
```

```
            break;
        }
        job = dequeue_job();
        pthread_mutex_unlock(&mutex);
        execute_job(job);
    }
    return NULL;
}
```

Graceful Shutdown

```
// Signal all workers to stop
pthread_mutex_lock(&mutex);
shutdown = true;
pthread_cond_broadcast(&cond);  // Wake ALL
pthread_mutex_unlock(&mutex);

// Wait for all threads to finish
for (int i = 0; i < num_threads; i++)
    pthread_join(threads[i], NULL);

// Clean up resources
pthread_mutex_destroy(&mutex);
pthread_cond_destroy(&cond);
```

Golden Rules

1. **Always** use `while` with `cond_wait`
2. **Lock** before checking/modifying shared state
3. **Unlock** what you lock (same thread!)
4. **Join** or **detach** every thread created
5. **Destroy** every resource you initialize
6. **Don't** pass stack variables without sync
7. **Lock in consistent order** to avoid deadlock
8. **Minimize** critical section duration

Compilation

```
# Basic compilation (Linux/macOS)
gcc -pthread program.c -o program

# With debugging and thread sanitizer
gcc -pthread -g -fsanitize=thread \
    program.c -o program

# With additional libraries
gcc -pthread program.c -o program -lm -lrt
```

Quick Reference

Thread Operations	
Create thread	`pthread_create`
Wait for threac	`pthread_join`
Exit thread	`pthread_exit`
Mutex Operations	
Lock (blocking	`pthread_mutex_lock`
Lock (try)	`pthread_mutex_trylock`
Unlock	`pthread_mutex_unlock`
Condition Variables	
Wait for signal	`pthread_cond_wait`
Signal one	`pthread_cond_signal`
Signal all	`pthread_cond_broadcast`
RW Locks & Barriers	
Read lock	`pthread_rwlock_rdlock`
Write lock	`pthread_rwlock_wrlock`
Barrier wait	`pthread_barrier_wait`

`#include <pthread.h>` | Link: `-pthread` | POSIX 1003.1c-1995